

FORMATION EMBARQUÉE

Module 02 — C++

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 02 — C++ : la puissance des objets, appliquée à l'embarqué

Le C++ est un « C avec des super-pouvoirs » : classes, RAII, templates, bibliothèque standard. Arduino est en réalité du C++. L'enjeu en embarqué : profiter des abstractions **sans coût caché** (le principe « zero-cost abstraction »).

Prérequis : module 01 (tout le C reste valable en C++). Pratique : `g++ main.cpp -o main -Wall -Wextra -std=c++17`.

1. Ce que C++ ajoute immédiatement au C

```
#include <iostream>

int main() {
    int n = 3;
    std::cout << "Valeur : " << n << "\n";    // E/S typées au lieu de printf

    bool ok = true;                            // vrai type booléen
    auto x = 42u;                               // déduction de type (x est unsigned)
    int& ref = n;                               // référence : alias sûr (jamais nul)
    ref = 5;                                    // n vaut 5
}
```

- **Références** (`int&`) : comme un pointeur toujours valide et sans `*`. Passage de gros objets sans copie : `void f(const Mesure& m)`.
- **Surcharge de fonctions** : plusieurs `afficher(int)`, `afficher(float)`.
- **Paramètres par défaut** : `void init(uint32_t baud = 115200);`
- `nullptr` remplace `NULL`, `constexpr` remplace beaucoup de macros :

```
constexpr uint32_t F_CPU = 16'000'000;        // calculé à la compilation
constexpr uint32_t ms_en_ticks(uint32_t ms) { return ms * (F_CPU / 1000); }
static_assert(ms_en_ticks(1) == 16000);       // vérifié à la compilation !
```

2. Classes et objets

Une classe regroupe **données + fonctions** qui les manipulent, avec un contrôle d'accès.

```

class Led {
public:
    Led(uint8_t broche) : broche_(broche), allumee_(false) { // constructeur
        pinMode(broche_, OUTPUT);
    }

    void allumer() { digitalWrite(broche_, HIGH); allumee_ = true; }
    void eteindre() { digitalWrite(broche_, LOW); allumee_ = false; }
    void basculer() { allumee_ ? eteindre() : allumer(); }
    bool estAllumee() const { return allumee_; } // const : ne modifie rien

private:
    uint8_t broche_; // inaccessible de l'extérieur → invariants protégés
    bool allumee_;
};

Led statut(13);
statut.basculer();

```

Concepts clés : - **Encapsulation** : `private` protège l'état interne ; l'extérieur passe par l'interface `public` .
- **Constructeur** : garantit qu'un objet naît dans un état valide. La **liste d'initialisation** (: `broche_(broche)`) initialise avant le corps. - **Destructeur** `~Led()` : appelé automatiquement à la destruction — la clé du RAI (§3). - `this` : pointeur vers l'objet courant. - **struct vs class** : identiques, sauf visibilité par défaut (public vs private).

3. RAI : l'idée la plus importante du C++

Resource Acquisition Is Initialization : une ressource (mémoire, verrou, périphérique, fichier) est acquise dans le constructeur et **libérée automatiquement** dans le destructeur. Plus d'oubli possible.

```

class SectionCritique {
public:
    SectionCritique() { noInterrupts(); } // acquisition
    ~SectionCritique() { interrupts(); } // libération GARANTIE
};

void copier_compteur(uint32_t& dst, volatile uint32_t& src) {
    SectionCritique sc; // interruptions coupées ici
    dst = src;
} // ← réactivées ici, même si exception/return anticipé

```

En embarqué, RAI sert pour : sections critiques, chip-select SPI, mutex RTOS, transactions I2C. C'est l'argument n°1 pour passer de C à C++.

4. Héritage et polymorphisme

4.1 Héritage

```
class Capteur {                                // classe de base
public:
    virtual ~Capteur() = default;              // destructeur virtuel si héritage !
    virtual int16_t lire() = 0;                 // = 0 → méthode "pure" : classe abstraite
    const char* nom() const { return nom_; }
protected:
    explicit Capteur(const char* nom) : nom_(nom) {}
private:
    const char* nom_;
};

class CapteurTemp : public Capteur {
public:
    CapteurTemp() : Capteur("temp") {}
    int16_t lire() override {                  // override : le compilateur vérifie
        return lire_adc() / 4;
    }
};
```

4.2 Polymorphisme dynamique

```
Capteur* capteurs[] = { &temp, &pression, &humidite };
for (Capteur* c : capteurs) {
    log(c->nom(), c->lire());    // la BONNE méthode est appelée pour chaque type
}
```

Sous le capot : une **vtable** (table de pointeurs de fonctions) — coût modeste (une indirection + un pointeur par objet) mais réel. En embarqué on l'utilise volontiers pour les **drivers interchangeables** (même interface, matériels différents), et on l'évite dans les chemins ultra-critiques.

4.3 Alternative sans coût : le polymorphisme statique (templates)

```
template <typename Bus>
class EcranOled {
public:
    explicit EcranOled(Bus& bus) : bus_(bus) {}
    void pixel(uint8_t x, uint8_t y) { bus_.ecrire(/*...*/); }
private:
    Bus& bus_;
};

EcranOled<BusI2c> ecran(i2c);    // résolu à la COMPILATION : zéro indirection
```

5. Templates et bibliothèque standard

5.1 Fonctions et classes génériques

```
template <typename T>
constexpr T borner(T v, T mini, T maxi) {
    return v < mini ? mini : (v > maxi ? maxi : v);
}
borner<int16_t>(t, -400, 850);

template <typename T, size_t N>
class TamponCirculaire { // taille fixée à la compilation : zéro malloc
public:
    bool put(const T& v) { /* ... */ }
    bool get(T& v) { /* ... */ }
private:
    T buf_[N];
    size_t tete_ = 0, queue_ = 0;
};
TamponCirculaire<uint8_t, 64> rx_uart;
```

5.2 STL : quoi utiliser en embarqué

OK sur micro	À éviter sur petit micro (allocation dynamique)
<code>std::array<T,N></code>	<code>std::vector</code> , <code>std::string</code> , <code>std::map</code>
<code>std::optional</code> , <code>std::pair</code>	<code>std::function</code> (peut allouer)
<code><algorithm></code> (<code>std::min</code> , <code>std::sort</code> , <code>std::clamp</code>)	flux <code>iostream</code> (très lourds)
<code>std::atomic</code> (si supporté)	exceptions et RTTI (souvent désactivés : <code>-fno-exceptions</code> <code>-fno-rtti</code>)

Sur Linux embarqué (Raspberry Pi, i.MX...), toute la STL est utilisable.

6. C++ moderne utile (C++11 → C++17)

```
auto v = lire_capteur(); // déduction de type

for (auto& mesure : historiques) { /* ... */ } // boucle "range-based"

enum class Mode : uint8_t { Repos, Mesure, Erreur }; // enum typée, sans fuite de noms
Mode m = Mode::Mesure;

// Lambdas : fonctions anonymes – parfaites en callback
bouton.surAppui([]() { led.basculer(); });

auto seuil = 100;
capteurs.filtrer([seuil](int16_t v) { return v > seuil; }); // capture de seuil

// Gestion mémoire automatique (Linux embarqué / gros systèmes)
#include <memory>
auto driver = std::make_unique<DriverSpi>(config); // libéré automatiquement
```

Et les mots-clés de robustesse : `override`, `final`, `= delete` (interdire la copie d'un driver : `Led(const Led&) = delete;`), `explicit` (interdire les conversions implicites du constructeur).

7. Différences C / C++ qui piègent

1. C++ est plus strict sur les conversions de pointeurs (`void*` → cast explicite requis).
2. `struct` / `enum` : plus besoin de `typedef`.
3. Pour appeler du C depuis du C++ (drivers constructeur, HAL) :

```
extern "C" {
#include "stm32f1xx_hal.h" // empêche le "name mangling" C++
}
```

1. Les objets globaux ont des **constructeurs exécutés avant** `main()` — attention si le constructeur touche au matériel pas encore initialisé.
2. `new` / `delete` remplacent `malloc` / `free` (et appellent les constructeurs/destructeurs) — mêmes réserves qu'en C sur les petits micros.

8. Exemple complet : bouton anti-rebond + LED, en classes

```
#include <stdint>

class BoutonDebounce {
public:
    explicit BoutonDebounce(uint8_t broche, uint16_t delai_ms = 20)
        : broche_(broche), delai_(delai_ms) {
        pinMode(broche_, INPUT_PULLUP);
    }

    // À appeler à chaque tour de boucle. Renvoie true sur UN appui détecté.
    bool appuye(uint32_t maintenant_ms) {
        bool brut = (digitalRead(broche_) == LOW);
        if (brut != dernier_brut_) {           // changement → on relance le chrono
            t_changement_ = maintenant_ms;
            dernier_brut_ = brut;
        }
        if ((maintenant_ms - t_changement_) > delai_ && brut != etat_stable_) {
            etat_stable_ = brut;               // état confirmé
            return etat_stable_;              // front d'appui uniquement
        }
        return false;
    }

private:
    uint8_t broche_;
    uint16_t delai_;
    bool dernier_brut_ = false;
    bool etat_stable_ = false;
    uint32_t t_changement_ = 0;
};

BoutonDebounce bouton(2);
Led led(13);

void loop() {
    if (bouton.appuye(millis())) led.basculer();
}
```

Remarque : aucun `delay()` — le programme reste réactif. Ce style « non bloquant » est développé au module 03.

9. Quand choisir C ou C++ ?

Situation	Choix
Micro 8 bits minuscule, driver très bas niveau	C
Projet Arduino / ESP32 / STM32 applicatif	C++ (sous-ensemble raisonnable)
Base de code d'équipe, drivers interchangeable	C++ (interfaces, RAII)
Linux embarqué, application riche	C++ complet (ou Rust, module 06)
Contribution au noyau Linux, code hérité	C

Le « C++ embarqué raisonnable » : classes, RAII, templates, `constexpr`, références — mais pas d'exceptions, pas de RTTI, pas d'allocation dynamique imprévisible.

Exercices

1. Réécris le tampon circulaire du module 01 en classe template `TamponCirculaire<T, N>` avec tests.
2. Crée une interface abstraite `IAfficheur` (méthodes `effacer()`, `texte(x, y, str)`) et deux implémentations factices (console, "LCD"). Fais tourner le même code applicatif sur les deux.
3. Écris une classe RAII `ChipSelect` qui abaisse une broche CS dans le constructeur et la relâche dans le destructeur.
4. Transforme la FSM du feu tricolore (module 01) en classe avec la méthode `tick(uint32_t maintenant_ms)`.

→ Suite : **Module 03 — Arduino** : on branche du vrai matériel.